

//

The `lalign` Package

Jamie Fravel

jamiefravel@me.com

Corresponds to `lalign` v1.2, dated June 25th 2026

Abstract

This is a collection of macros that I developed while writing my dissertation. They are mostly focused around displaying, naming and referencing mathematical programming problems. The `lalign` environment macro formats math programs for legibility in both code and display. The named paragraph and sub-equations macros will be useful in any document which defines and references multiple problems, algorithms or multi-lined environments.

Contents

1	Installation	3
2	The <code>lalign</code> Environment	3
3	Named Paragraphs	7
4	Named Sub-equation	8
5	Combining Macros	10
6	Spacing Macros	11
7	Known Bugs	13
8	Feature Wishlist	14
9	Version History	14

1 Installation

Copy `lalign.sty` to the same directory as your main `.tex` file and add the command `\usepackage{lalign}` to your preamble. The `amsmath` and `expl3` packages are required but will also be added by the above command. I recommend using these macros in conjunction with the `hyperref` package so that custom tags/references are clickable.

2 The lalign Environment

`lalign` is an environment macro built on the `alignat` environment from the `amsmath` package which helps to format mathematical programming problems aesthetically so that they are legible in code as well as in the final pdf. Its primary function is to automatically embed the objective function in a zero-width box so that its width does not interfere with the constraints spacing or centering.

The `lalign` environment takes a few arguments:

- M. Objective Direction: Maximize, Minimize, max or min.
- M. Objective Function: the mathematical expression to be optimized over.
- O. The optional distance to raise the “s.t.” clause above vertical center of the first constraint. Defaults to `0pt`.
- O. The optional distance to raise the objective clause above vertical center of the first constraint. Defaults to `0pt`.
- B. Constraint Body: the constraints are given, in `alignat` format, between `\begin` and `\end`. Importantly, a double ampersand `&&` should be placed after every line break `\\` to preserve spacing.
- S. The `lalign*` environment is identical without printing equation reference tags unless explicitly called for by the `\tag` command.

```
\begin{lalign}{Maximize}{\mathbf{c}^\top\mathbf{x}}
  A\mathbf{x} \leq \mathbf{b} \\&
  \mathbf{x} \geq \mathbf{0} \\
\end{lalign}
```

↓

$$\text{Maximize } \mathbf{c}^\top \mathbf{x} \tag{1}$$

$$\text{s.t. } A\mathbf{x} \leq \mathbf{b} \tag{2}$$

$$\mathbf{x} \geq \mathbf{0} \tag{3}$$

The ampersand `&` controls alignment; the constraint rows will be aligned at the ampersand.

$Ax + s = b + 0$	$Ax + s \leq b + 0$	$Ax + s = b + 0$
$x, s \geq 0$	$x, s \geq 0$	$x, s \geq 0$

↓

$\max c^\top x$	$\max c^\top x$	$\max c^\top x$
$\text{s.t. } Ax + s = b + 0$	$\text{s.t. } Ax + s = b + 0$	$\text{s.t. } Ax + s = b + 0$
$x, s \geq 0$	$x, s \geq 0$	$x, s \geq 0$

Indexed Constraints

Indexed constraints are also possible. An additional double-ampersand `&&` will properly space and align a for-all $\forall$ statement.

```
\begin{lpalign*}{Maximize}{\sum_{i=1}^I c_i x_i}
\mathbf{a}_i^\top \mathbf{x} \leq b_i
&& \forall i \in [I]
x_i \geq 0 && \forall i \in \{1, 2, \dots, I\}
\end{lpalign*}
```

↓

$$\begin{aligned} &\text{Maximize } \sum_{j=1}^J c_j x_j \\ &\text{s.t. } \mathbf{a}_i^\top \mathbf{x} \leq b_i \quad \forall i \in [I] \\ &\quad x_i \geq 0 \quad \forall i \in \{1, 2, \dots, I\} \end{aligned}$$

`amsmath`'s `aligned` environment can be used to format too-long objectives or constraints. Equation tags will be placed correctly.

```

\begin{lpalign}{Maximize}
{
\begin{aligned}
c_1&x_1+c_2x_2+c_3x_3+c_4x_4\\
&+c_5x_5+\dots+c_Jx_J
\end{aligned}
}
\begin{aligned}
a_{i1}&x_1+a_{i2}x_2+a_{i3}x_3\\
&+a_{i4}x_4+\dots+a_{IJ}x_J \leq b_i
\end{aligned}
&& \text{forall } i \in [I] \\
\mathbf{x} &\geq \mathbf{0}
\end{aligned}
\end{lpalign}

```

↓

$$\begin{array}{ll} \text{Maximize} & c_1x_1 + c_2x_2 + c_3x_3 + c_4x_4 \\ & + c_5x_5 + \dots + c_Jx_J \end{array} \quad (4)$$

$$\begin{array}{ll} \text{s.t.} & a_{i1}x_1 + a_{i2}x_2 + a_{i3}x_3 \\ & + a_{i4}x_4 + \dots + a_{IJ}x_J \leq b_i \end{array} \quad \forall i \in [I] \quad (5)$$

$$\mathbf{x} \geq \mathbf{0} \quad (6)$$

Clause Alignment

Some may think that the placement of “s.t.” is a little bit awkward in the above example. It can be raised or lowered with the use of the optional third argument.

```

\begin{lpalign}{...}{...}
...
\end{lpalign}

\begin{lpalign}{...}{...}[4\straise][-4\objraise]
...
\end{lpalign}

```

↓

$$\begin{array}{ll}
 \max & \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix}^\top \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \\
 \text{s.t.} & \begin{bmatrix} a_{11} & \dots & a_{1n} \\ a_{21} & \dots & a_{2n} \\ \vdots & \vdots & \vdots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}
 \end{array}
 \qquad
 \begin{array}{ll}
 \max & \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix}^\top \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \\
 \text{s.t.} & \begin{bmatrix} a_{11} & \dots & a_{1n} \\ a_{21} & \dots & a_{2n} \\ \vdots & \vdots & \vdots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}
 \end{array}$$

The lengths `\straise` and `\objraise` are $1/4\text{\baselineskip}$ so that, if the corresponding vector has n entries, `n\straise` should be about halfway above vertical center of the matrix.

Default raising lengths—in addition to the spacing between the constraints and indexing statement—can be altered via the following commands:

- `\setlength{\straisedefault}{0pt}`
- `\setlength{\objraisedefault}{0pt}`
- `\setlength{\forallspace}{2em}`

```
\setlength{\forallspace}{0.5em}
```

$$\begin{array}{ll}
 \text{Maximize} & \sum_{j=1}^J c_j x_j \\
 \text{s.t.} & \mathbf{a}_i^\top \mathbf{x} \leq b_i \quad \forall i \in [I] \\
 & x_i \geq 0 \quad \forall i \in \{1, 2, \dots, I\}
 \end{array}$$

```
\setlength{\forallspace}{10em}
```

$$\begin{array}{ll}
 \text{Maximize} & \sum_{j=1}^J c_j x_j \\
 \text{s.t.} & \mathbf{a}_i^\top \mathbf{x} \leq b_i & \forall i \in [I] \\
 & x_i \geq 0 & \forall i \in \{1, 2, \dots, I\}
 \end{array}$$

Tags and Labels

Use the `\tag` and `\label` commands to customize tags and to reference particular constraints. `\notag` will mute a particular tag. Neither `\tag` nor `\notag` should interfere with the numbering of other equation tags.

```
\begin{lpalign}{Maximize}{c^{\top} x \quad \label{obj}}
  Ax+s &= b \quad \tag{Cons}\label{cons} \quad \&\&
  s &\geq 0 \quad \notag \quad \&\&
  x &\geq 0 \quad \label{nneg}
\end{lpalign}
```

`\eqref{obj}` is the objective function.
`\eqref{cons}` encodes the body constraints.
 Non-negativity is enforced by `\eqref{nneg}`.
 The slack constraints are not tagged.

↓

$$\text{Maximize } c^{\top} x \tag{7}$$

$$\text{s.t. } Ax + s = b \tag{Cons}$$

$$s \geq 0$$

$$x \geq 0 \tag{8}$$

[\(7\)](#) is the objective function. [\(Cons\)](#) encodes the body constraints. Non-negativity is enforced by [\(8\)](#). The slack constraints are not tagged.

3 Named Paragraphs

The `paratitle` command displays a paragraph marker that takes

- O. an optional extra indent.
- M. a title.
- O. an optional abbreviation.

If the paragraph is labeled, then any reference to it will give the abbreviation rather than a counter number.

```
\paratitle{Named Paragraph}[NPg]\label{npara} \\
Paragraph \ref{npara} is a named paragraph.
```

↓

Named Paragraph (NPg)
Paragraph NPg is a named paragraph.

The first optional argument is a sort of preamble. It is intended to be used for custom indent spacing.

```
\paratitle[\HSKP]{Named Paragraph}[NPg]\label{npara2} \\
Paragraph \ref{npara2} is a named paragraph.
```

↓

Named Paragraph (NPg)
Paragraph NPg is a named paragraph.

4 Named Sub-equation

The `namedsubeqs` environment will use a given name as the first index in any enclosed equation reference tag. A second index is appended as in a traditional `subequations` environment. Options are given as key-value pairs in the optional argument. The currently supported keys are `style` for the second-index formatter (default `\alph`) and `punct` for the separator between the main tag and the sub-tag (default `.`). For backward compatibility, a single optional argument without an `=` sign is still interpreted as the style argument.

- M. a name or abbreviation.
- O. optional key-value settings such as `style=\roman` or `punct=-`.


```

\begin{namedsubeqs}{EQ}
\begin{align}
A\mathbf{x} &= \mathbf{b} & \label{eq:1} & \\
C\mathbf{y} &= \mathbf{d} & \label{eq:2} & \\
\end{align}
\end{namedsubeqs}

```

Equations `\ref{eq:1}` and `\ref{eq:2}` are equations.

↓

$$Ax = b \quad (EQ.a)$$

$$Cy = d \quad (EQ.b)$$

Equations [EQ.a](#) and [EQ.b](#) are equations.

Other common tag styles include `\roman` and `\arabic`.

```

\begin{namedsubeqs}{EQ}[style=\roman]

\begin{namedsubeqs}{EQ}[style=\arabic]

```

↓

$$Ax = b \quad (EQ.i) \quad Ax = b \quad (EQ.1)$$

$$Cy = d \quad (EQ.ii) \quad Cy = d \quad (EQ.2)$$

Fonts can be changed using the appropriate switch commands.

```

\begin{namedsubeqs}{EQ}[style=\itshape\roman]

\begin{namedsubeqs}{EQ}[style=\scshape\alph]

```

↓

$$Ax = b \quad (EQ.i) \quad Ax = b \quad (EQ.A)$$

$$Cy = d \quad (EQ.ii) \quad Cy = d \quad (EQ.B)$$

`punct` changes the separator between the main tag and the sub-tag.

```
\begin{namedsubeqs}{EQ}[style=\roman,punct=-]
```

↓

$$A\mathbf{x} = \mathbf{b} \quad (\text{EQ-i})$$

$$C\mathbf{y} = \mathbf{d} \quad (\text{EQ-ii})$$

Also available is `\subref` and `\subeqref` which work like `\ref` and `\eqref` but only reference the secondary, sub-index of the tag. Inside both `namedsubeqs` and traditional `subequations`, ordinary `\label` is enough. The `\sublabel` macro remains available.

```
\begin{namedsubeqs}{SQ}[style=\itshape\roman]
\begin{align}
A\mathbf{x} = \mathbf{b} \quad \label{sq:1} \\
C\mathbf{y} = \mathbf{d} \quad \label{sq:2}
\end{align}
\end{namedsubeqs}
```

We can reference both (`\subref{sq:1}`-`\subref{sq:2}`) or just `\subeqref{sq:1}` or even `\eqref{sq:2}`.

↓

$$A\mathbf{x} = \mathbf{b} \quad (\text{SQ.i})$$

$$C\mathbf{y} = \mathbf{d} \quad (\text{SQ.ii})$$

We can reference both (*[i-ii](#)*) or just (*[i](#)*) or even ([SQ.ii](#)).

`\subref` and `\subeqref` work with ordinary `\label` inside both `namedsubeqs` and traditional `subequations` environments.

$$A\mathbf{x} = \mathbf{b} \quad (16a)$$

$$C\mathbf{y} = \mathbf{d} \quad (16b)$$

Testing subrefs with subequations: `\ref{test}` [16](#), `\ref{test:1}` [16a](#), `\subref{test:1}` [a](#), `\subeqref{test:2}` ([b](#))

5 Combining Macros

Each of these macros may be combined for a nicely named, labeled, formatted and tagged mathematical programming problem

```

\paratitle{Non-Linear Program}[NLP]\label{para:NLP}
\begin{namedsubeqs}{NLP}
\begin{lalign}{Minimize}{f(\mathbf{x})} \label{NLP.0}}
g_i(\mathbf{x}) &\geq 0 \ \&\forall i \in I \label{NLP.1} \ \&\&
h_j(\mathbf{x}) &= 0 \ \&\forall j \in J \label{NLP.2} \ \&\&
\mathbf{x} &\geq 0 \label{NLP.3}
\end{lalign}
\end{namedsubeqs}

```

Problem \eqref{para:NLP} is a nonlinear program.
 Equations \eqref{NLP.0} is the objective function.
 Equations (\ref{NLP.1}-\ref{NLP.3}) are the constraints.
 Non-negativity is enforced by constraint \subeqref{NLP.3}.

↓

Non-Linear Program (NLP)

$$\begin{array}{ll}
 \text{Minimize } f(\mathbf{x}) & \text{(NLP.i)} \\
 \text{s.t. } g_i(\mathbf{x}) \geq 0 & \forall i \in I \quad \text{(NLP.ii)} \\
 h_j(\mathbf{x}) = 0 & \forall j \in J \quad \text{(NLP.iii)} \\
 \mathbf{x} \geq 0 & \text{(NLP.iv)}
 \end{array}$$

Problem (NLP) is a nonlinear program. Equations (NLP.i) is the objective function. Equations (NLP.ii-iv) are the constraints. Non-negativity is enforced by constraint (iv).

6 Spacing Macros

Quick Spacing

Also included are font-relative versions of the common spacing commands `\quad` and `\qquad` and a few more for good measure:

- | | | | | | |
|-------------------------------|--|--|----------------------------|--|--|
| • <code>\hskip = 0.5em</code> | | | • <code>\HSkp = 3em</code> | | |
| • <code>\Hskip = 1em</code> | | | • <code>\HskP = 4em</code> | | |
| • <code>\Hskp = 2em</code> | | | | | |

Negative versions of the above spacing macros:

- | | |
|-------------------------------|-----------------------------|
| • <code>\htrg = -0.5em</code> | • <code>\HTRg = -3em</code> |
| • <code>\Htrg = -1em</code> | • <code>\HTRG = -4em</code> |
| • <code>\HTrg = -2em</code> | |

Vertical versions of the above horizontal spacing macros:

- `\vskp` = 0.5em
- `\Vskp` = 1em
- `\VSkp` = 2em
- `\VSKp` = 3em
- `\VSKP` = 4em
- `\vtrg` = -0.5em
- `\Vtrg` = -1em
- `\VTrg` = -2em
- `\VTRg` = -3em
- `\VTRG` = -4em

These are, I would say, less reliable than using `\vspace` or `\vskip` correctly, but I find them more convenient.

Some horizontal spacing between these `\HSPK` words `\`

Some vertical spacing between these `\VSKp` lines `\`

Some negative-horizontal spacing between these `\HTrg` words `\`

Some negative-vertical space between these `\Vtrg` lines, so that they overlap.

↓

Some horizontal spacing between these words

Some vertical spacing between these

lines

Some negative-horizontal spacing between these words

Some negative-vertical space between these lines, so that they overlap.

Summation Spacing

Some macros for correctly spacing display-style summation notation when the lower index set is overfull. This is done by placing the subscript in a zero-width box and should not be used in inline math.

- `\SUM` = $\sum$
- `\PROD` = $\prod$
- `\CUP` = $\bigcup$
- `\CAP` = $\bigcap$
- `\VEE` = $\bigvee$
- `\WEDGE` = $\bigwedge$

These each take two arguments

- M. The lower index set
- O. The upper index

This spacing is a personal preference; I think it makes things more consistent and legible. The below example is comically exaggerated, please use `\substack` to illuminate any *really* long indexing.

$$\begin{array}{l}
 X=\sum_{i\text{\texttt{\textbackslash in way too much indexing}}} x_i \\
 X=\SUM{i\text{\texttt{\textbackslash in way too much indexing}}}x_i \\
 X=\SUM{\substack{i\text{\texttt{\textbackslash in way to \textbackslash\textbackslash much indexing}}}}x_i \\
 X=\SUM{i=1,300,000}[1,300,001]x_i \\
 \downarrow \\
 X = \sum_{i \in \text{\texttt{\textbackslash waytoomuchindexing}}} x_i \qquad X = \sum_{i \in \text{\texttt{\textbackslash waytoomuchindexing}}} x_i \\
 X = \sum_{\substack{i \in \text{\texttt{\textbackslash wayto} \\ \text{\texttt{\textbackslash muchindexing}}}}} x_i \qquad X = \sum_{i = 1,300,000}^{1,300,001} x_i
 \end{array}$$

Equation Spacing

A few binary relations with extra space on either side. I like these for aligned inequality constraints.

- `\EQ` $=$ • `\IN` $\in$ • `\NEQ` $\neq$ • `\NIN` $\notin$
- `\LS` $<$ • `\LE` $\leq$ • `\SUB` $\subset$ • `\SEQ` $\subseteq$
- `\GS` $>$ • `\GE` $\geq$ • `\SUP` $\supset$ • `\SPQ` $\supseteq$

$$\begin{array}{l}
 Ax+s \EQ b \EQ \\
 x,s \GE 0 \\
 \downarrow \\
 \max c^\top x \\
 \text{s.t. } Ax+s = b+0 \\
 x,s \geq 0
 \end{array}
 \qquad
 \begin{array}{l}
 Ax+s \EQ b \EQ \\
 x,s \GE 0 \\
 \downarrow \\
 \max c^\top x \\
 \text{s.t. } Ax+s = b+0 \\
 x,s \geq 0
 \end{array}$$

7 Known Bugs

- ☒ Spacing macros `\quad` and `\qqquad` overwrote default, absolute behavior with font-relative spacing. Fixed by renaming macros to `\Hskip` and `\Hskp`; v1.1.

- ☒ The package depended on `makecell` and `vcell` without using them. Fixed by removing those unused dependencies; v1.2.
- ☒ The `namedsubeqs` environment assigned `\@currentlabel` without switching to `\makeatletter` scope. Fixed internally; v1.2.
- ☒ The `\subref` and `\subeqref` macros failed outright when `hyperref` was not loaded. Fixed by falling back to plain text references when `hyperref` is absent; v1.2.
- ☒ When `hyperref` was loaded, `\sublabel` could produce duplicate equation destinations or occasionally link to the wrong tag. Fixed by assigning dedicated `hyperref`-facing equation identifiers and by removing a redundant equation-counter decrement at the end of `namedsubeqs`; test branch after v1.2.
- ☐ The objective function argument in the `lalign` environment is embedded in a zero-width box to avoid interfering with the spacing/alignment of the constraints. Unfortunately, this means that the objective function does not contribute to the width of environment object. So, if the objective function is long enough, the LP will not center properly or may even overflow the page. For now, use an embedded `aligned` environment to display a long function on multiple lines.
- ☒ Ordinary `\label` now works for `\subref` and `\subeqref` inside both `namedsubeqs` and traditional subequations. Fixed on the test branch after v1.2.

8 Feature Wishlist

- ☐ Enclose an `lalign` environment in a figure-like structure which will move to the next page without interrupting prose if space doesn't allow for it without a page break.
- ☐ Remove required double-ampersands `&&` from the end of lines in the `lalign` environment.

9 Version History

- v1.0 Dec. 20, 2025. Initial distribution. Includes `lalign` and `namedsubeqs` environments along with macros for named paragraphs and spacing.
- v1.1 Jan. 30, 2026. Bug fix. Renamed quick spacing macros to maintain compatibility with standard `TEX`. Added negative, vertical and negative-vertical versions. Added summation spacing macros. Added clause alignment arguments to the `lalign` environment.
- v1.2 Jun. 25, 2026. Bug fix. Removed unused package dependencies, fixed the internal `\@currentlabel` handling in `namedsubeqs`, added a no-`hyperref` fallback for `\subref` and `\subeqref`, and cleaned up documentation inconsistencies.